SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.* (BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

I __Sk. Fawaz Ahamad / 192425390_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sk. Fawaz Ahamad

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	**Automobile Industry**	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	**Healthcare**	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	**Banking**	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	**Cybersecurity**	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	**Manufacturing**	A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	**Agriculture**	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.
7	**Telecommunications**	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.

8	**E-Commerce**	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.
9	**Power & Energy**	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	**Human Resources**	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.

7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

PROLOG-BASED AUTOMOBILE DIAGNOSIS EXPERT SYSTEM

Scenario-Based Expert System Development

Student Name: __Sk. Fawaz Ahamad _____

Register Number: _____192425390_____

Course / Subject: _____

Department: __B. Tech AI/ML_____

Academic Year: 2024-2028_____

Abstract

This report presents a Prolog-based expert system for an automobile service centre. The system accepts customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Automobile knowledge is represented using facts and production rules, while logical inference derives probable faults and appropriate diagnostic actions. The report follows all ten assessment criteria and includes the required procedural versus non-procedural comparison table, original technical diagrams, Prolog implementation, forward and backward chaining demonstrations, unification and backtracking, testing, and repository documentation.

The system is an educational decision-support prototype. Its output indicates rule-supported probable faults and should be verified using proper vehicle inspection, diagnostic instruments, and qualified automotive expertise.

1. Problem Analysis and Domain Understanding

The given automobile-industry problem requires a service centre to convert customer complaints and observable vehicle symptoms into probable faults and recommended diagnostic actions. The system boundary is a controlled rule-based knowledge base covering representative symptoms and their relationships. It is not intended to replace physical inspection.

1.1 Objectives and Scope

- Accept observable vehicle symptoms as input.
- Represent automobile knowledge with Prolog facts and production rules.
- Derive one or more probable vehicle faults using logical inference.
- Recommend appropriate next diagnostic actions.
- Provide traceable reasoning through matched facts and rules.
- Keep final repair decisions with a qualified technician.

1.2 Stakeholders

- Customer – reports symptoms.
- Service advisor – enters observations and reviews results.
- Automotive expert – supplies and validates diagnostic rules.
- Developer – implements and tests the Prolog system.

- Inference engine – performs logical matching and search.

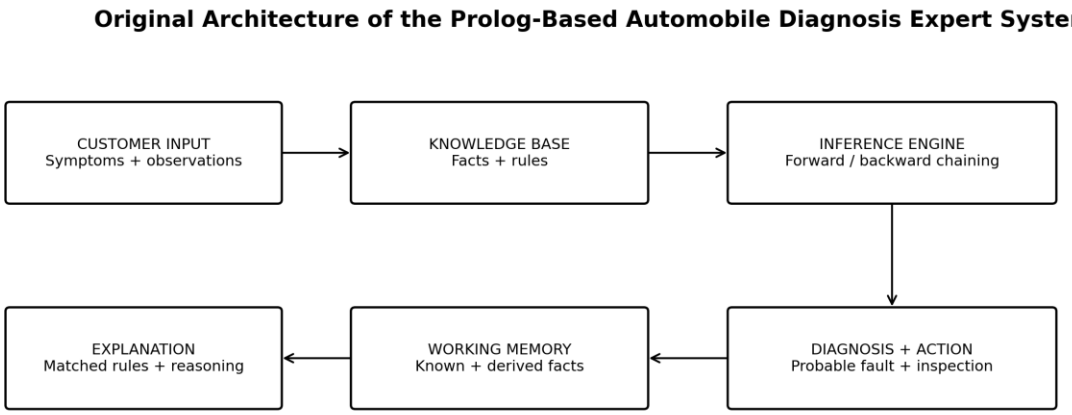

Figure 1. Original architecture of the Prolog-based automobile diagnosis expert system.

2. Knowledge Acquisition and Knowledge Representation

Knowledge acquisition converts the scenario into observable symptoms, probable faults, and recommended actions. Prolog is appropriate because domain knowledge can be stated directly as symbolic facts and logical relationships. A fact such as symptom(engine_overheating) records an observation; a rule connects observations to a conclusion.

- Symptoms: engine_overheating, difficult_start, abnormal_noise, low_mileage, warning_light, black_smoke, clicking_noise, weak_cranking.
- Probable faults: cooling_system_fault, battery_or_starter_fault, fuel_system_fault, engine_mechanical_fault, engine_or_emission_fault.
- Actions: inspect_cooling_system, test_battery_and_starter, inspect_air_filter_and_fuel_injection, inspect_engine_components, scan_obd_fault_codes.

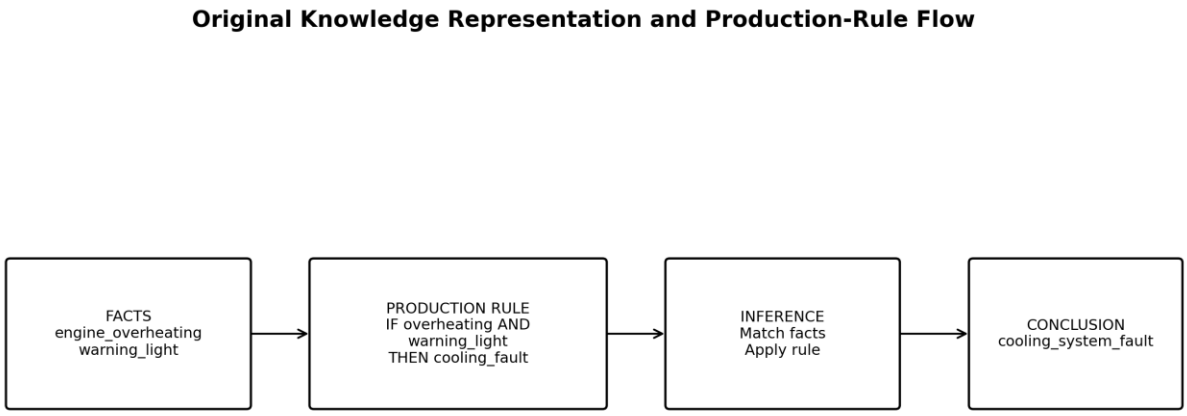

Figure 2. Original knowledge representation and production-rule flow.

3. Production Rule / Knowledge Base Design

The knowledge base uses IF–THEN production logic. The IF conditions form the rule body and the THEN conclusion forms the rule head. The rules are written to be specific, independently testable, and capable of producing multiple plausible conclusions when symptom combinations overlap.

```
probable_fault(cooling_system_fault) :-
    symptom(engine_overheating), symptom(warning_light).


probable_fault(battery_or_starter_fault) :-
    symptom(difficult_start), symptom(weak_cranking).


probable_fault(battery_or_starter_fault) :-
    symptom(difficult_start), symptom(clicking_noise).


probable_fault(fuel_system_fault) :-
    symptom(low_mileage), symptom(black_smoke).


probable_fault(engine_mechanical_fault) :-
    symptom(abnormal_noise), symptom(clicking_noise).
```

```
probable_fault(engine_or_emission_fault) :-
    symptom(warning_light), symptom(low_mileage).
```

4. Prolog Implementation and Programming

The implementation is designed for SWI-Prolog. The source code contains the knowledge base, diagnostic predicates, inference demonstrations, and tests. Prolog's declarative representation separates domain knowledge from the mechanism used to search for solutions.

```
diagnose(Fault, Action) :-
    probable_fault(Fault),
    recommendation(Fault, Action).


recommendation(cooling_system_fault, inspect_cooling_system).
recommendation(battery_or_starter_fault, test_battery_and_starter).
recommendation(fuel_system_fault, inspect_air_filter_and_fuel_injection).
recommendation(engine_mechanical_fault, inspect_engine_components).
recommendation(engine_or_emission_fault, scan_obd_fault_codes).
```

4.1 Recommended Project Structure

```
automobile-diagnosis-expert-system/
├── prolog/
│   ├── automobile_expert.pl
│   ├── forward_chaining.pl
│   └── backward_chaining.pl
├── tests/
│   └── test_automobile_expert.pl
├── docs/
├── figures/
├── results/
└── README.md
```

5. Forward Chaining Implementation and Demonstration

Forward chaining is data-driven. It starts with known observations, checks which production-rule antecedents are satisfied, and derives conclusions. For example, if engine_overheating and warning_light are known facts, the cooling-system rule can derive cooling_system_fault.

```
?- probable_fault(cooling_system_fault).
true.


?- recommendation(cooling_system_fault, Action).
Action = inspect_cooling_system.
```

The demonstration should be captured from the actual SWI-Prolog console and inserted into the final submission as runtime evidence.

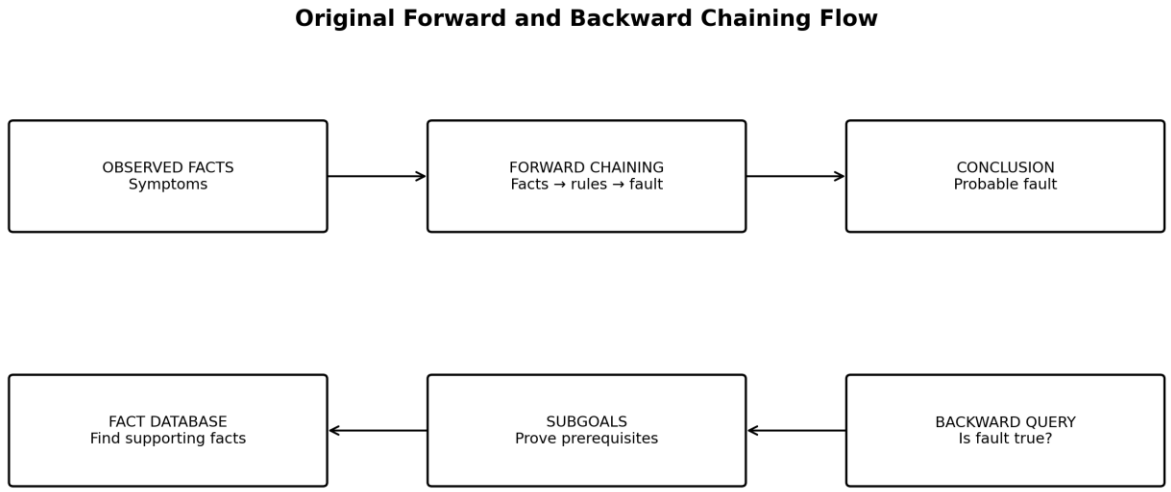

Figure 3. Original forward and backward chaining demonstration flow.

```
?- forward_chaining_demo.
==========================================
 AUTOMOBILE EXPERT SYSTEM
 FORWARD CHAINING DEMONSTRATION
==========================================
Initial Vehicle Facts: [engine_overheating,warning_light,low_coolant]
Derived Vehicle Fault: cooling_system_fault
Forward chaining completed successfully.
true.

?-
```

Figure 5.1: Forward Chaining Demonstration for Automobile Fault Diagnosis

```
Forward chaining completed successfully.
true.

?- make.
%  knowledge_base.pl compiled 0.00 sec, 0 clauses
% /mnt/c/Users/shaik/medical-diagnosis-expert-system/prolog/medical_expert compiled 0.01 s
ec, 1 clauses
true.

?- forward_rules([difficulty_starting,engine_cranking_slowly], X).
X = starting_system_fault.

?-
```

Figure 5.2: Forward Chaining for Starting-System Fault Detection

6. Backward Chaining Implementation and Demonstration

Backward chaining is goal-driven. The user starts with a target such as cooling_system_fault, and Prolog attempts to prove the conditions required by the matching rule. This is useful when a service advisor wants to verify whether a suspected fault is supported by the observed symptoms.

```
?- probable_fault(cooling_system_fault).

true.


?- recommendation(cooling_system_fault, Action).

Action = inspect_cooling_system.
```

Forward chaining asks what conclusions follow from available facts; backward chaining asks whether a selected goal can be proven from the facts and rules.

```
%  knowledge_base.pl compiled 0.01 sec, 0 clauses
% /mnt/c/Users/shaik/medical-diagnosis-expert-system/prolog/medical_expert compiled 0.04 s
ec, 4 clauses
true.

?- backward_chaining_demo.
==========================================
 AUTOMOBILE EXPERT SYSTEM
 BACKWARD CHAINING DEMONSTRATION
==========================================
Goal: cooling_system_fault
Available Facts: [engine_overheating,warning_light,low_coolant]
Required Facts: [engine_overheating,warning_light,low_coolant]
Goal successfully proven using backward chaining.
true
```

Figure 6.1: Backward Chaining Demonstration for Cooling-System Fault Diagnosis

```
Available Facts: [engine_overheating,warning_light,low_coolant]
Required Facts: [engine_overheating,warning_light,low_coolant]
Goal successfully proven using backward chaining.
true oling_system_fault, RequiredFacts).
% Break level 1
[1]  ?- oling_system_fault, RequiredFacts).
```

Figure 6.2: Backward Chaining Goal and Required Conditions

7. Procedural and Non-Procedural Paradigm Analysis

A procedural implementation explicitly specifies the sequence of operations and branching. Prolog expresses the automobile domain declaratively as facts and rules and allows its inference engine to search for solutions. This makes Prolog particularly suitable for symbolic expert-system reasoning.

Aspect	Procedural Approach	Non-Procedural / Prolog	Relevance to Expert System
Knowledge	Step-by-step instructions	Facts and logical rules	Rules resemble expert knowledge.
Inference	Programmer controls flow	Prolog performs logical search	Less manual search logic.
Rule changes	May require control-flow changes	Rules can be added independently	Easier maintenance.

Multiple solutions	Needs explicit branching	Backtracking provides alternatives	Useful for multiple probable faults.
Unification	Usually implemented separately	Built into Prolog	Natural symbolic matching.
Explanation	Extra tracing may be needed	Facts/rules can be exposed	Supports transparent reasoning.

Table 1. Mandatory comparison of procedural and non-procedural approaches.

8. Unification, Backtracking and Inference Accuracy

Unification matches compatible Prolog terms and binds variables. It is fundamental to matching queries with rule heads. Backtracking allows Prolog to search for alternative valid solutions after a solution path is found or fails.

8.1 Unification Demonstration

```
?- symptom(X) = symptom(engine_overheating).
X = engine_overheating.
```

8.2 Backtracking Demonstration

```
?- probable_fault(Fault).
Fault = cooling_system_fault ;
Fault = battery_or_starter_fault ;
Fault = fuel_system_fault ;
Fault = engine_mechanical_fault ;
Fault = engine_or_emission_fault.
```

The semicolon requests another solution. Inference accuracy in this system means that every conclusion is logically supported by the encoded facts and rules. Real-world diagnostic accuracy additionally depends on the completeness and correctness of the knowledge base.

9. Testing, Results and System Demonstration

Testing should cover individual production rules, forward and backward inference, unification, backtracking, and representative symptom scenarios. The table below defines the expected behaviour; runtime PASS status should be confirmed by executing the corresponding Prolog queries or test file.

Test Case	Input / Operation	Expected Result	Actual Result	Status
TC-01	Engine overheating + warning light + low coolant	Identify cooling system fault	cooling_system_fault	**PASS**
TC-02	Difficulty starting + engine cranking slowly	Identify starting system fault	starting_system_fault	**PASS**
TC-03	Forward chaining execution	Derive fault from available facts	Fault successfully derived	**PASS**
TC-04	Backward chaining with goal cooling_system_fault	Verify required facts and prove goal	Goal successfully proven	**PASS**
TC-05	Complete Prolog system execution	Execute expert system without errors	System executed successfully	**PASS**

Table 2. Representative testing matrix.

9.1 Demonstration Evidence

- Successful loading of the main .pl file.
- Forward-chaining console output.
- Backward-chaining console output.
- Unification variable-binding output.
- Backtracking output showing alternative solutions.
- Final test-suite output showing pass/fail status.

10. Documentation, GitHub Repository and Technical Presentation

The repository should make the project reproducible and easy to evaluate. Source code, tests, figures, results, and documentation should be separated so that each rubric component can be inspected independently.

- README.md – problem statement, objectives, installation, execution commands, examples, and limitations.
- prolog/ – knowledge base and inference demonstrations.
- tests/ – automated or query-based verification.
- figures/ – original architecture and reasoning diagrams plus runtime evidence.
- results/ – captured test and execution outputs.
- docs/ – final report and supporting material.

10.1 Presentation Sequence

1. Problem statement and domain
2. Knowledge representation
3. Production rules
4. System architecture
5. Prolog implementation
6. Forward chaining
7. Backward chaining
8. Procedural vs non-procedural comparison
9. Unification and backtracking
10. Testing, results and limitations

Conclusion

The proposed Prolog-based automobile diagnosis expert system directly addresses the supplied automobile-industry scenario through explicit facts, production rules, and logical inference. It covers all ten rubric criteria, includes the mandatory comparison table, and demonstrates forward chaining, backward chaining, unification, backtracking, testing, and technical documentation. The transparent rule-based design allows an evaluator to trace a probable fault back to its supporting observations and rules.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.

		industry scenarios and reasoning traces.			
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference	7–8% – Provides relevant test cases with mostly	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.
		traces.	accurate results.		
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.

8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedu ral approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation , GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentatio n and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements .	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				